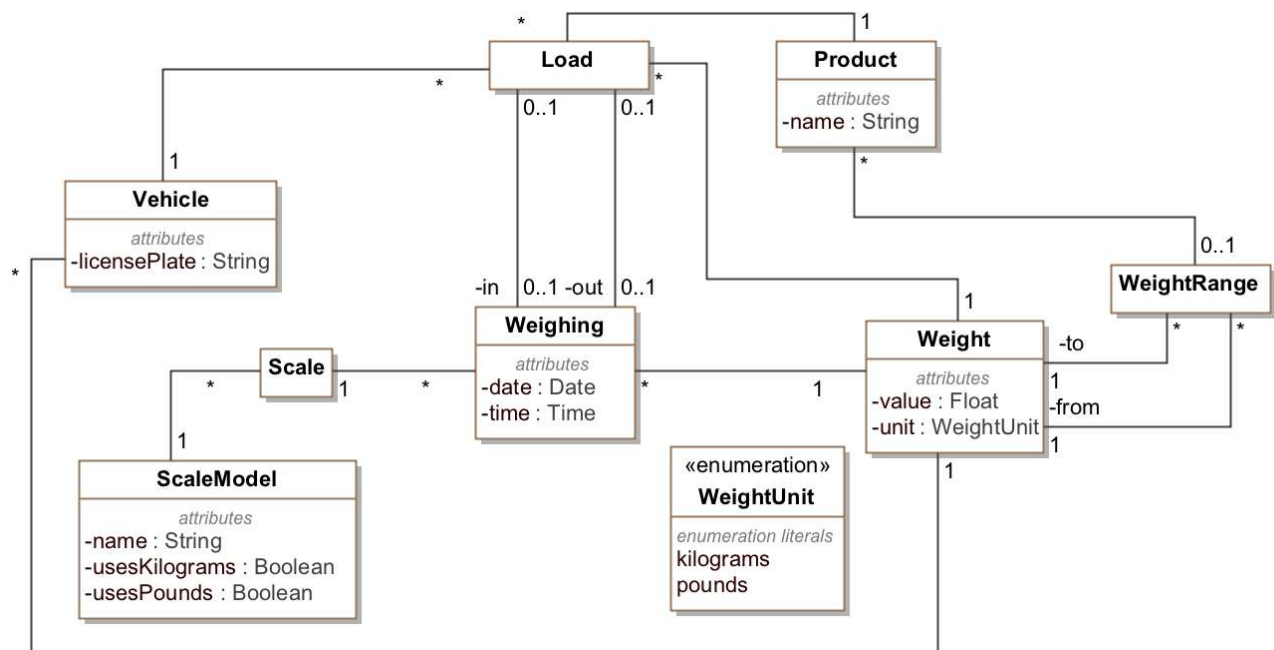


Análisis y Diseño con Patrones

Práctica 2: Patrones de diseño y de asignación de responsabilidades

Para poder elaborar esta Práctica partiremos de la solución de la Práctica 1.

Recuerda que nos han contratado para desarrollar un software de control de mercancías que llegan al almacén del mercado central de una ciudad. El diagrama de clases resultante de la práctica 1 era el siguiente:



Restricciones de integridad:

- El peso de un cargamento con pesaje de entrada y de salida debe ser igual a la diferencia entre los dos pesajes (con un margen de error del 5%).
- El peso de salida de un vehículo debe ser mayor o igual que su tara (que es el peso del vehículo sin personal de servicio, pasajeros ni carga y con la dotación completa de agua, combustible, etc.).
- Un modelo de báscula debe tener *usesKilograms* o *usesPounds* a true y sólo uno de los dos puede ser true.

Observaciones:

- El peso de un cargamento siempre está en kilogramos.

Ejercicio 1 (20%)

Supongamos ahora que tenemos una operación *getReading()* en la clase *Scale* que devuelve una instancia de tipo *Weight* que corresponde a la lectura actual de la báscula.

Nos proponemos diseñar una operación (la llamaremos *createWeighing* pero no es necesario que se llame así en su solución) que cree una instancia de *Weighing* que, a partir de una instancia de *Scale* y una de *Load* debe hacer lo siguiente:

- Obtener la lectura de la báscula
- Guardar el peso leído como peso del pesaje
- Guardar la fecha actual (con la operación *Date.today()* que devuelve la fecha actual)
- Guardar la hora actual (con la operación *Time.now()* que devuelve la hora actual)
- Guardar una referencia a la báscula (por la asociación con la clase *Scale*)
- Guardar una referencia a la carga (por la asociación con *Load*)
- Invocar la operación *addWeighing* de la instancia de *Load* (esta operación recibe la instancia de *Weighing* que estamos creando como parámetro) para que la carga asocie este pesaje como pesaje de entrada o de salida según sea necesario

- ¿A qué clase corresponde la responsabilidad de implementar la operación *createWeighing()*?
- ¿A qué clase corresponde la responsabilidad de determinar si el pesaje es de entrada o de salida? (consideramos que es de entrada si no hay ningún pesaje de entrada anterior asociado a la carga y es de salida si ya hay un pesaje de entrada asociado).
- Diseña la operación *createWeighing()*. Puedes usar pseudocódigo o un diagrama de secuencia. Tal y como hemos dicho en el enunciado, que la clase *Scale* ya tiene una operación llamada *getReading()* que devuelve la lectura actual.
 - Puedes suponer que la clase *Load* tiene una operación llamada *addWeighing*, que recibe una instancia de *Weighing* y lo asocia a la carga como entrada o salida según toque.
 - Puedes suponer que la clase *Scale* no se guarda ninguna referencia a las instancias de *Weighing* que tiene asociadas.

Solución:

a)

Aplicando el patrón de asignación de responsabilidades Creador, las dos clases candidatas creador serían *Scale* y *Load*. Como de las responsabilidades descritas se puede deducir que habrá acoplamiento de *Load* a *Weighing* pero no que haya de *Scale* a *Weighing*, añadiríamos la operación *createWeighing* a la clase *Load* ya que es la que añade menos acoplamiento.

b)

Aplicando el patrón Experto, la clase indicada para esta responsabilidad es la clase *Load* ya que es la que tiene la información necesaria.

c)

```
public class Load {

    public void createWeighing (Scale scale)

    {Weighing weighing = new Weighing(Date.today(), Time.now (), scale, this);

        this.addWeighing(weighing);

    }

}

Public class Weighing {

    private final Date date;

    private final Time time;

    private Load load;

    private Scale scale;

    private Weight weight;

    public Weighing (Date date, Time time, Scale scale, Load load){

        this.date = date;

        this.time = time;

        this.load = load;

        this.weight = scale.getReading ();

    }

}
```

```
this.scale = scale;
```

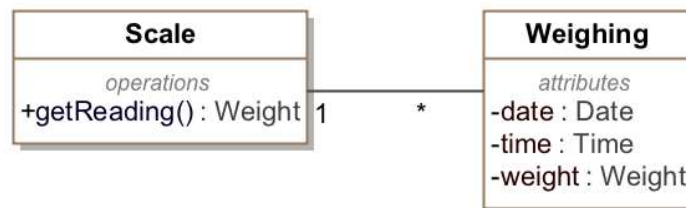
```
}}
```

Ejercicio 2 (20%)

Supongamos ahora que tenemos que incorporar un nuevo tipo de báscula que funciona con un software algo diferente. Supongamos también que la clase que implementa este modelo tiene la siguiente interfaz:

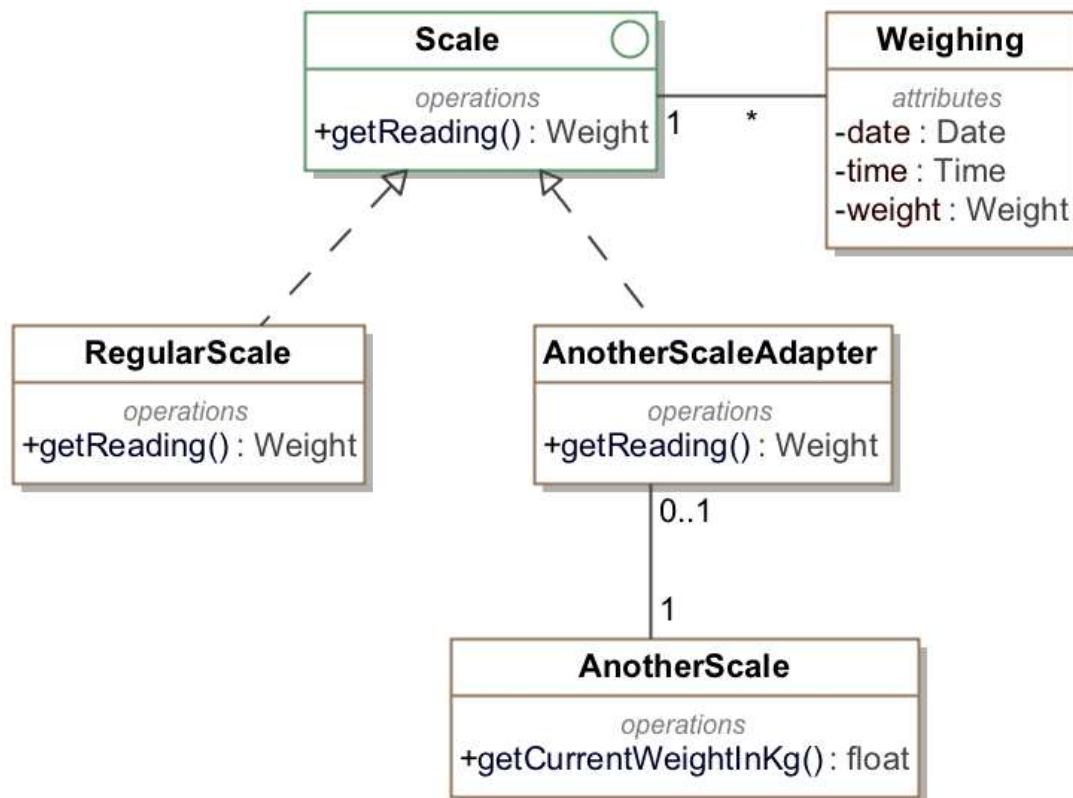


Queremos incorporar este tipo de báscula con el mínimo impacto sobre el código que ya está funcionando como, por ejemplo, la operación del ejercicio anterior. Qué patrón de diseño es adecuado en esta situación? Realiza el diagrama de clases resultante de añadir la nueva clase y aplicar este patrón a este diagrama de partida:



Solución

El patrón que soluciona este problema es el patrón Adaptador, ya que nos permite operar con las instancias de *AnotherScale* como si fueran instancias de *Scale*:



Hemos tenido que introducir una interfaz “*Scale*” (y cambiar el nombre de la clase *Scale* por *RegularScale* para mantener el mismo nombre de clase a los objetos que utilizan la interfaz).

Ejercicio 3 (20%)

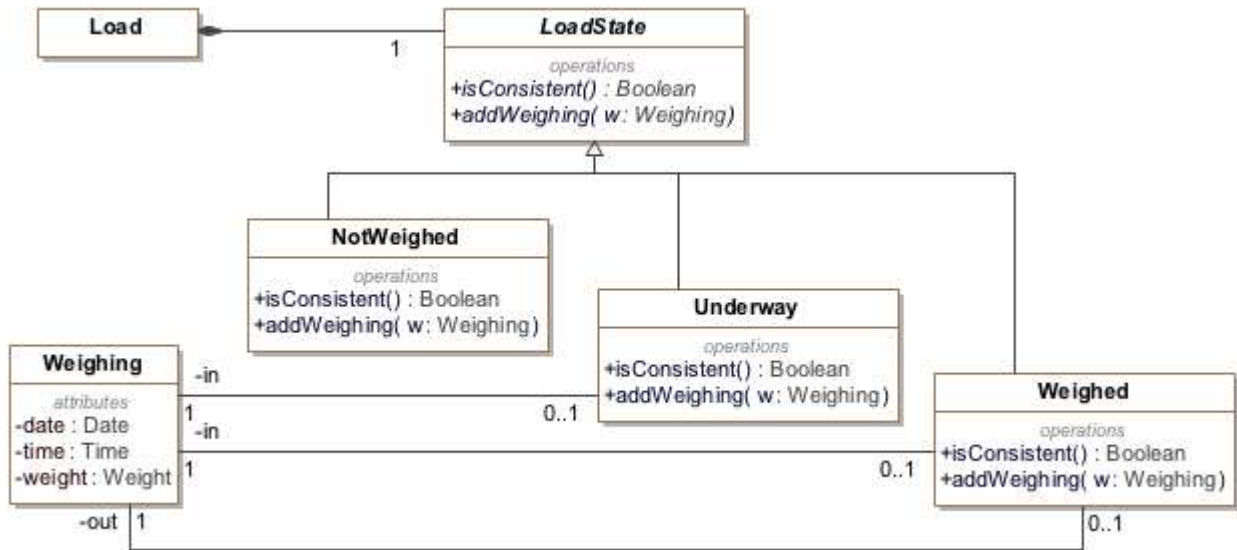
La clase *Load* tiene un comportamiento diferente en función de si tiene pesaje de entrada y / o salida. En concreto, queremos centrarnos en dos operaciones:

- *isConsistent()*
 - Devuelve true si la carga está "no pesada" (no tiene pesaje de entrada)
 - Retorna true si la carga está "en curso" (tiene pesaje de entrada pero no de salida).
 - Devuelve true si la carga está "pesada" (tiene los dos pesajes) y la tara del vehículo es menor o igual que el peso del pesaje de salida
 - devuelve falso en otro caso
 - *addWeighing()* (lo hemos visto en el ejercicio 1)
 - Si la carga está "no pesada" lo asocia al pesaje de entrada
 - Si la carga está "en curso" lo asocia al pesaje de salida
 - Si la carga está "pesada" muere un error
- a) Qué patrón de diseño es adecuado aplicar en esta situación, con el fin de diseñar estas operaciones?
- b) Diseña la operación *addWeighing()*. Puedes usar pseudocódigo o un diagrama de secuencia.

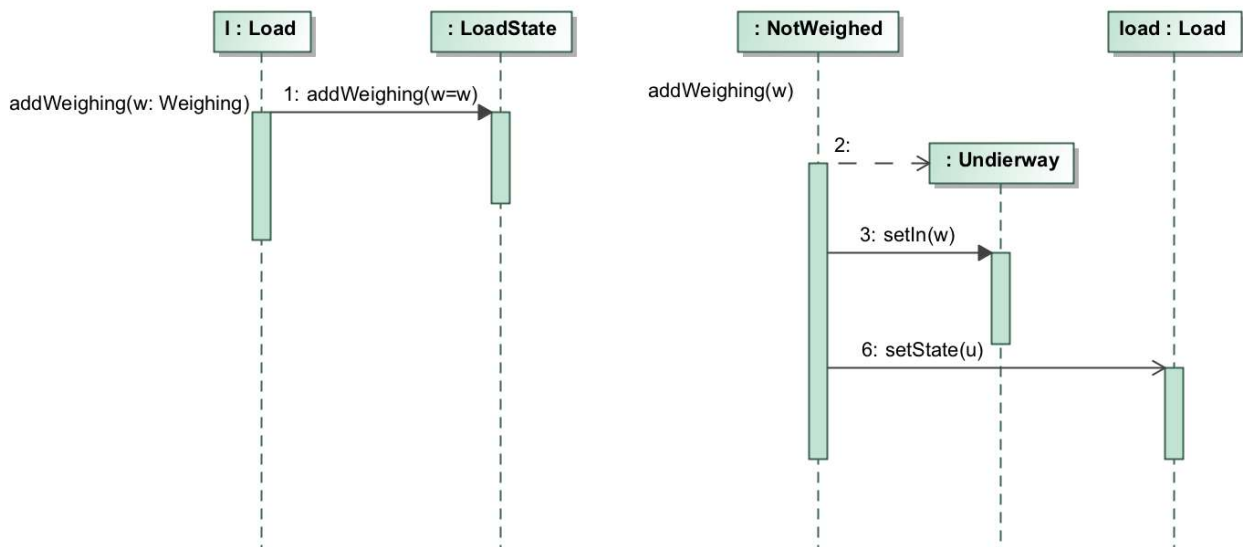
Solución

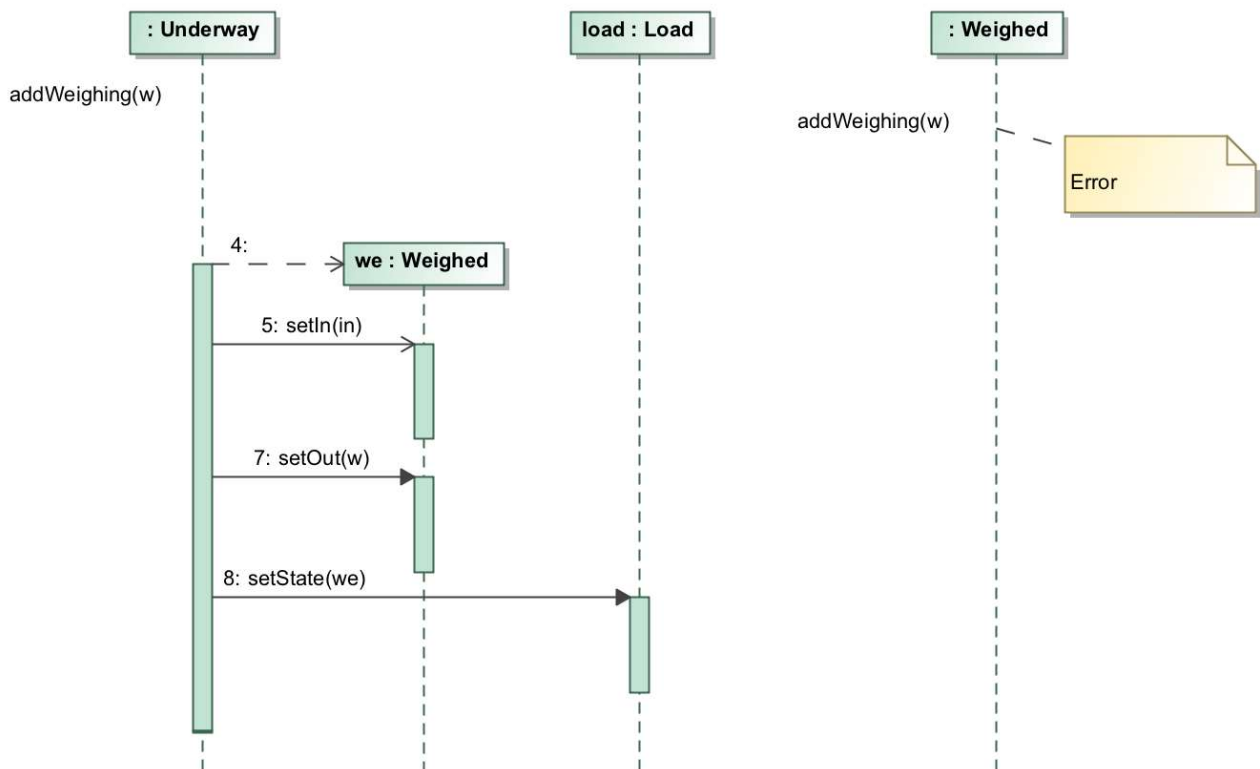
- a) Si consideramos que "no pesada", "en curso" y "pesada" son estados de la carga, entonces el patrón adecuado sería el patrón Estado ya que ambas operaciones deben comportarse de manera diferente en función del estado.
- b)

En este caso, el diagrama de clases quedaría de la siguiente manera:



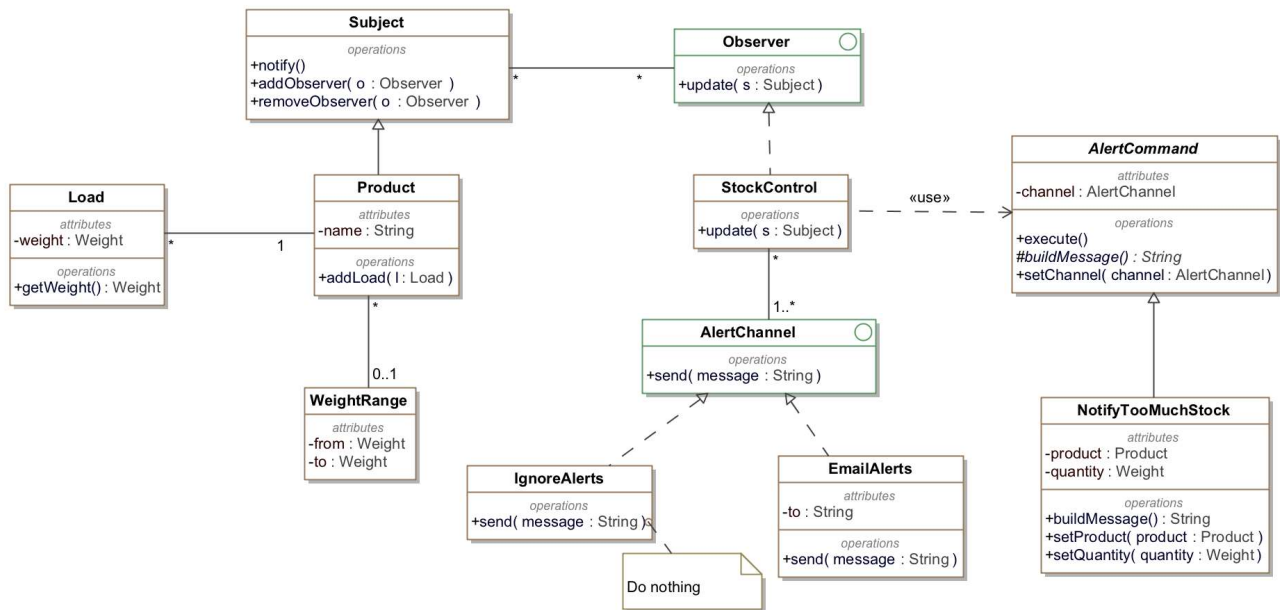
y los diagramas de secuencia:





Ejercicio 4(20%)

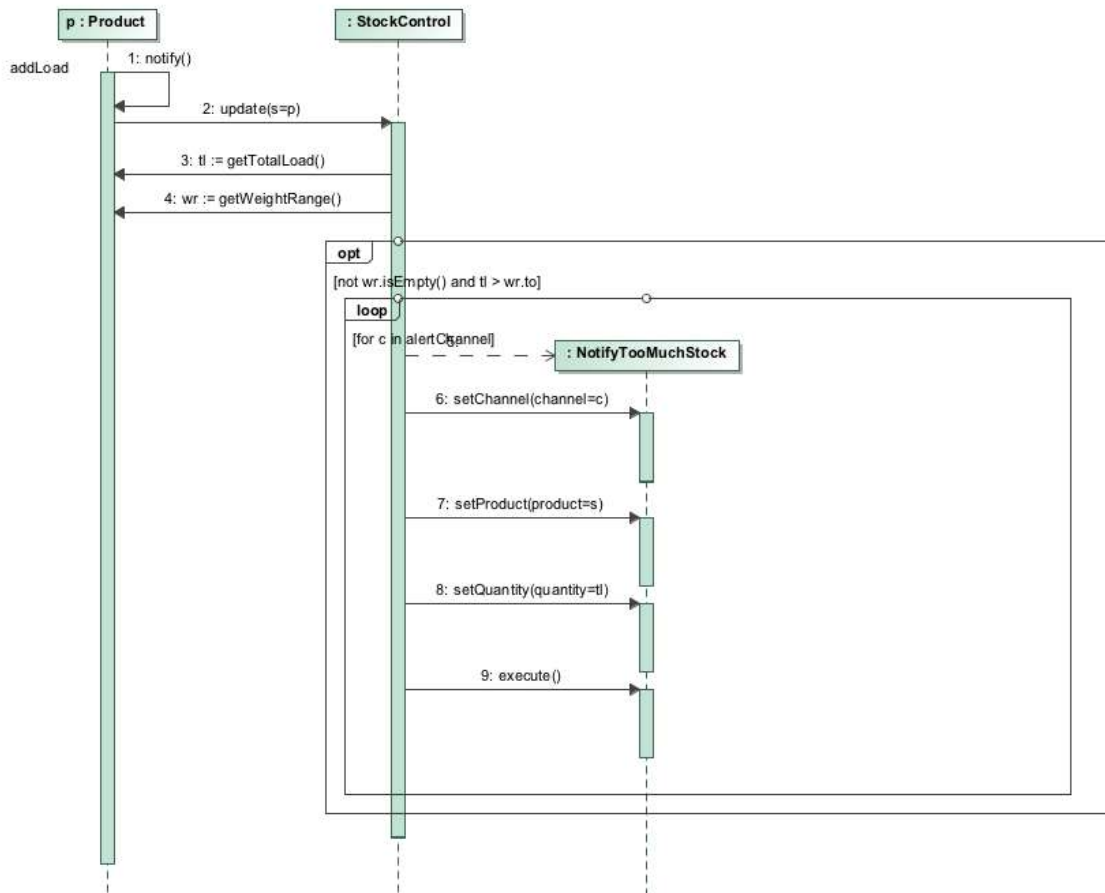
Supondremos ahora que estamos diseñando el sistema que controla que no haya más producto de la cuenta de un tipo determinado. Supongamos una nueva clase *StockControl* que se encarga de enviar unas notificaciones cuando se produce la situación en que se ha recibido una carga de un producto que sobrepasa el límite de producto que podemos almacenar. El diagrama de clases es el siguiente:



- ¿Qué patrones se han aplicado en este diagrama de diseño? (indica, como mucho, 3)
- Proponer el diagrama de secuencia o escriba el pseudocódigo que muestre cómo se resuelve la operación `addLoad()` de la clase `Product` suponiendo que existe una operación `getTotalLoad` a la clase `Product` que nos devuelve la suma los pesos de todas las cargas de un producto.

Solución

- "Command" en la clase `NotifyTooMuchStock`
 - "Observer" `Product` y `StockControl`
 - "Null Object" en `IgnoreAlerts`
-



Ejercicio 5(20%)

En el ejercicio anterior, el subsistema de alertas nos ha quedado un poco complicado ya que, por ejemplo, para enviar una notificación de que hay demasiado stock de un producto debemos hacer lo siguiente:

1. Crear una instancia de *NotifyTooMuchStock*.
2. Asignar el canal de comunicación.
3. Asignar el producto.
4. Asignar el peso.
5. Invocar al método *execute* de *NotifyTooMuchStock*.

Nos gustaría que la clase *StockControl* viera una versión simplificada de este subsistema y que, de paso, se redujera el acoplamiento hacia el mismo.

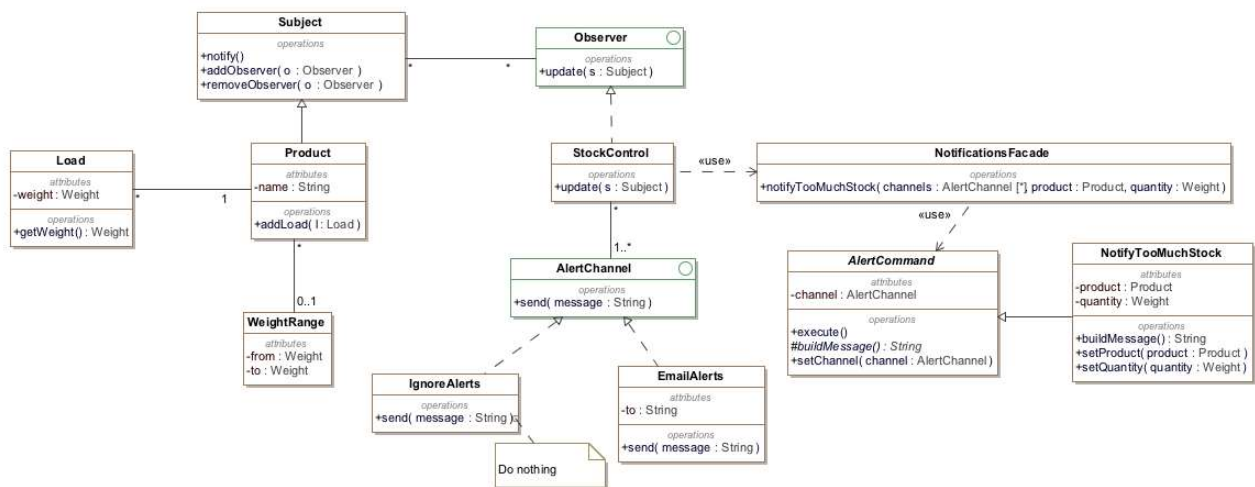
- a) ¿Qué patrón crees que es adecuado aplicar en esta situación?

- b) Realiza el diagrama de clases resultante de aplicar este patrón.
- c) Elabora el diagrama de secuencia o el pseudocódigo de la operación *notifyTooMuchStock(p: Product, stock: Weight)* de la clase *StockControl* (no es necesario incluir la lógica de validación de si se debe enviar o no la notificación, sólo el envío de la alerta utilizando la visión simplificada del subsistema de alertas).

Solución

- a) El patrón que soluciona el problema descrito es el patrón "Fachada"

b)



c)

